Department of Computer Science and Engineering
Computer Network Lab (CS39006)

Assignment 4: Implementing a Network Application with Concurrent Servers (“Campus QuickPrint” – A Network Print Queue Service)

In this assignment, You will implement a network-based print management system where multiple clients connect to a central print server to:

- Submit print jobs
- Check job status
- List submitted jobs
- Cancel pending jobs
- Disconnect gracefully

The server must handle multiple clients concurrently using `fork()`.

Server Program (`quickprint_server.c`):

The server program must create a TCP socket, bind it to a port specified via command line argument, and listen for incoming client connections. It should run continuously, accepting new connections in an infinite loop. Whenever a client connects, the server must immediately create a new child process using `fork()` to handle that client independently, while the parent process continues listening for additional clients. The child process must close the listening socket, communicate with the connected client, process all protocol commands, maintain job-related data for that session, and terminate cleanly when the client disconnects or issues a quit command. The parent process must close the connected socket descriptor after forking and continue accepting new connections. The implementation must ensure that terminated child processes do not remain as zombie processes.

Client Program (`quickprint_client.c`):

The client program must create a TCP socket and connect to the server using the IP address and port provided through command line arguments. It should provide an interactive command-line interface where users can enter commands. The client must send properly formatted protocol messages to the server and display server responses in a readable format. The client should terminate gracefully when the user issues the quit command.

Protocol Design Requirements:

You are required to design the application-layer protocol. The protocol must clearly define the structure of request messages and response messages, including how multi-line data (such as document content for submission) is transmitted and how messages are delimited or framed. You must define status codes for successful and erroneous operations, specify the format of job identifiers, and clearly describe how errors are communicated to the client. The complete protocol specification must be documented as comments within your code.

Required Operations:

Your application must support the following commands:

- `HELLO <username>` – Establishes a session with the server and registers the client's username for the current connection.
- `SUBMIT <title>` – Submits a new print job with the given title and associated multi-line document content, and returns a unique job ID.
- `STATUS <jobid>` – Requests the current state (QUEUED, PRINTING, DONE, or CANCELED) of the specified job.
- `LIST` – Displays all jobs submitted during the current session along with their IDs and statuses.
- `CANCEL <jobid>` – Cancels the specified job if it has not already completed.
- `QUIT` – Terminates the client session and closes the connection gracefully.

The server must simulate realistic job progression. When a job is submitted, it should initially be marked as `QUEUED`, then change to `PRINTING` after a short delay (approximately 10 seconds), and finally change to `DONE` after another short delay (approximately 20 seconds). This may be implemented using timestamps and time comparisons.

Implementation Guidelines:

1. **Implementing data structures:**
   a. A fixed-size array `jobs[20]` is enough.
   b. Each job:
      `int id; char title[...]; enum status;`
      `time_t submitted_at; int canceled;`
   c. Maintain `next_job_id` per session (starting at 1).
   d. For status simulation, don't use extra threads. Compute status dynamically using elapsed time since `submitted_at` (unless canceled).

2. **Implement Commands in this order:**
   a. `HELLO:` store username in a variable and respond with welcome.
   b. `SUBMIT:` parse title from the first line, then read subsequent lines until you get some delimiter (say, a full stop). Store the text (optional) or ignore body after reading it (simpler) but still must consume it from socket correctly. Create a job record with a timestamp and return JOBID.
   c. `STATUS:` find job by id; if not found return some error code like 404. If found and not canceled, compute state based on elapsed seconds (0 - 10 queued, 10 - 30 printing, >30 done). Note, you can define your own timeout values. This is just a guideline.
   d. `LIST:` iterate jobs and print one job per line in the response.
   e. `CANCEL:` if job exists and not DONE, mark canceled and return success; if DONE, return an error like `409 ALREADY_DONE`.
   f. `QUIT:` send a goodbye response and break loop to close socket.

3. **How to handle zombies?**
   At server startup, do:

```
#include <signal.h>

int main() {
      // tells the OS to auto-reap children on exit (on Linux)
      signal(SIGCHLD, SIG_IGN);
      ...
}
```

   This avoids zombies without writing a custom handler.

4. **How to handle the time values?**

For the “QUEUED → PRINTING → DONE” simulation, the cleanest approach is to store a timestamp when the job is submitted, and then compute the status on demand whenever the client asks STATUS or LIST.

Store at submit time:

```
#include <time.h>

typedef enum { QUEUED, PRINTING, DONE, CANCELED } job_state_t;

typedef struct {
    int id;
    char title[128];
    job_state_t state;
    struct timespec submitted_at; // monotonic timestamp at submit
    int canceled;
} job_t;

static double elapsed_seconds (struct timespec start, struct timespec now) {
    double s = (double)(now.tv_sec - start.tv_sec);
    double ns = (double)(now.tv_nsec - start.tv_nsec) / 1e9;
    return s + ns;
}
```

When creating a job:

```
clock_gettime(CLOCK_MONOTONIC, &jobs[i].submitted_at);
jobs[i].canceled = 0;
```

Compute status dynamically:

```
job_state_t compute_state(const job_t *j) {
    if (j->canceled) return CANCELED;

    struct timespec now;
    clock_gettime(CLOCK_MONOTONIC, &now);
    double dt = elapsed_seconds(j->submitted_at, now);

    if (dt < 10.0) return QUEUED;
    if (dt < 30.0) return PRINTING;
```

```
    return DONE;
}
```

Important tips: Don't `sleep(10)` inside the child to "advance" the job, because that blocks handling other commands from that client. Always prefer timestamp-based computation so the server remains responsive.

You can check Linux man page section 3 for time related functions. `man 3 time`